

Parallel Computing — NVIDIA AI Infrastructure

What is Parallel Computing?

Parallel computing is a computing technique in which a large problem is divided into multiple smaller tasks that are executed **simultaneously** using multiple processing resources. Instead of waiting for one operation to finish before starting the next, many operations can be performed at the same time.

The easiest way to remember it is:

Serial Computing → one task after another

Parallel Computing → many tasks at the same time

For example, imagine processing **1,000 images** with an AI model. In serial computing, images could be processed one after another. In parallel computing, many images or many mathematical operations within those images can be processed concurrently across multiple compute units.

Parallel computing is fundamental to modern AI because neural networks perform enormous numbers of **matrix, vector, and tensor operations**, many of which can be calculated independently and simultaneously.

CUDA is NVIDIA's platform for accelerated computing and the foundation for GPU computing.

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model created by [Nvidia](#) that allows software to use graphics processing units (GPUs) for general-purpose mathematical and scientific tasks.

Instead of GPUs being used only for graphics, CUDA enables applications to execute workloads such as **machine learning, deep learning, scientific computing, simulations, and high-performance computing** on NVIDIA GPUs.

NVIDIA parallel computing is a system that allows computers to run many calculations at the exact same time. Instead of processing tasks one after another like a traditional CPU, it uses the massive power of a Graphics Processing Unit (GPU) to break big problems into thousands of smaller pieces and solve them all at once.

How It Works

- **Massive Threads:** GPUs contain thousands of smaller cores designed to handle separate tasks simultaneously rather than focusing on a single fast core.
- **The CUDA Platform:** NVIDIA introduced [CUDA](#) in 2006 as a programming model and software toolkit. It lets developers write code in standard languages like C++ to direct instructions straight to the GPU.

- **Heterogeneous Computing:** The CPU handles serial tasks—like opening files or loading web pages—while handing heavy, repetitive math over to the GPU.

Why It Matters

- **Artificial Intelligence:** Modern AI and deep learning rely heavily on matrix multiplication and linear algebra. NVIDIA's parallel architecture accelerates training for large language models and neural networks.
- **Scientific Research:** Fields like fluid dynamics, molecular dynamics, and weather forecasting use parallel computing to run complex simulations fast.
- **Ready-to-Use Libraries:** Developers do not always have to write low-level code. NVIDIA provides optimized math libraries (like cuBLAS) that handle parallel work under the hood.

Serial Computing vs Parallel Computing

In **serial computing**, instructions are primarily executed sequentially. If a workload consists of tasks A, B, C, and D, the processor performs:

A → B → C → D

In **parallel computing**, independent portions of the workload can execute concurrently:

A + B + C + D → executed simultaneously

This does not mean that every program can be made completely parallel. Some operations depend on the results of previous operations and therefore must remain sequential.

For AI infrastructure, the key principle is:

The more independent mathematical work that can be executed simultaneously, the more effectively the workload can benefit from parallel hardware such as GPUs.

Advantages of Parallel Computing Over Serial Computing

The primary advantage of parallel computing is **higher computational throughput**. When thousands of operations can execute simultaneously, large workloads can finish significantly faster than when the same operations are performed sequentially.

Parallel computing also enables **scalability**. If a workload becomes too large for one processor or GPU, it may be distributed across multiple GPUs or even multiple servers. This is essential for modern large-scale AI training, where models and datasets may require clusters containing hundreds or thousands of accelerators.

Another important advantage is the ability to process **very large datasets and models**. Deep-learning training involves billions or trillions of mathematical operations. Performing these operations serially would make many modern AI workloads impractically slow.

Parallel computing can therefore provide:

Higher throughput → faster processing → larger workloads → scalable AI systems.

Major Types of Parallelism

For AI infrastructure, four types are particularly important: **data parallelism, model parallelism, tensor parallelism, and pipeline parallelism**.

Data Parallelism

Data parallelism means placing copies of the same model on multiple GPUs and giving each GPU a different portion of the training data.

For example:

GPU 1 → Model Copy → Batch A

GPU 2 → Model Copy → Batch B

GPU 3 → Model Copy → Batch C

GPU 4 → Model Copy → Batch D

Each GPU performs computations independently on its assigned data. During distributed training, the results—particularly gradients—must then be synchronized so the model replicas remain consistent.

Data parallelism is useful when the **model fits within the memory of each GPU**, but the training dataset or computational workload is large.

Certification memory rule:

Data Parallelism = Same model, different data.

Model Parallelism

Model parallelism means dividing a model across multiple GPUs because the entire model cannot efficiently fit or execute on a single GPU.

For example:

Large AI Model

↓ split across

GPU 1 + GPU 2 + GPU 3 + GPU 4

Instead of every GPU containing a complete model copy, different portions of the model are distributed across GPUs.

This becomes particularly important for very large neural networks where **model parameters, activations, optimizer states, and other runtime data** create memory requirements beyond the capacity of one accelerator.

Certification memory rule:

Model Parallelism = One model distributed across multiple GPUs.

Tensor Parallelism

Tensor parallelism is a form of model parallelism where individual mathematical operations, particularly large matrix or tensor operations, are divided across multiple GPUs.

Suppose a neural-network layer contains a very large matrix multiplication. Instead of performing the entire multiplication on one GPU, portions of the matrix can be distributed across several GPUs.

Conceptually:

Large Matrix Operation

↓ split

GPU 1 → Matrix Part 1

GPU 2 → Matrix Part 2

GPU 3 → Matrix Part 3

GPU 4 → Matrix Part 4

The partial results are then combined or exchanged as required.

Tensor parallelism is particularly relevant to **large language models (LLMs)** because individual transformer layers can contain extremely large matrix operations.

Pipeline Parallelism

Pipeline parallelism is a distributed training technique that splits a deep learning model's sequential layers into vertical segments or "stages," assigning each stage to a different device like a GPU so large models can run.

How Pipeline Parallelism Works

- **Vertical Sharding:** The model layers are divided into consecutive chunks, where each device holds only a fraction of the total network instead of duplicating the entire model.
- **Micro-batching:** A mini-batch of data is split into smaller **micro-batches** so that multiple devices can process data concurrently like an assembly line.

- **Forward and Backward Passes:** Each stage performs forward and backward computations, passing activations and gradients to neighboring stages via point-to-point send and receive operations.

How GPUs Use Parallel Computing

GPUs are specifically designed to exploit large amounts of parallelism.

A CPU typically contains a relatively small number of sophisticated cores optimized for general-purpose computation, control logic, and low-latency execution. A GPU provides a much larger amount of parallel arithmetic execution capacity.

When an AI workload contains thousands or millions of similar mathematical operations, the GPU divides that work into many small units that can execute concurrently.

Consider a simplified vector addition:

A = [1, 2, 3, 4]

B = [5, 6, 7, 8]

The result is:

C = A + B

A serial conceptual approach would calculate:

1+5 → 2+6 → 3+7 → 4+8

A parallel approach can conceptually calculate:

1+5 | 2+6 | 3+7 | 4+8

at approximately the same time across different execution resources.

Real AI workloads perform this type of parallel operation at an enormous scale.

SIMT — Single Instruction, Multiple Threads

SIMT stands for Single Instruction, Multiple Threads. It is the core execution model used in modern Graphics Processing Units (GPUs) (like those from NVIDIA and AMD) to handle massive parallel processing.

Think of SIMT as a way for a processor to run the same instruction across thousands of things at once, but with much more freedom than older hardware designs

NVIDIA GPUs organize threads into groups called **warps**. Threads within a warp execute instructions together using the GPU's SIMT execution model.

This architecture is highly efficient when threads are performing similar operations on different pieces of data.

Conceptually:

Same operation + many data elements → many threads → parallel GPU execution

This is one reason matrix and tensor calculations map extremely well onto GPUs.

How SIMT Works

- **Grouped Execution:** GPUs group threads into small packs—called **warps** (NVIDIA) or **wavefronts** (AMD)—typically containing 32 or 64 threads.
- **Shared Control:** The hardware fetches and decodes a single instruction for the whole group at one time.
- **Independent Identity:** Unlike older models, every single thread in that group keeps track of its own register state, stack, and **program counter** (the marker for where it is in the code).

SIMT vs. SIMD (Single Instruction, Multiple Data)

While SIMT sounds a lot like SIMD (used in CPU vector extensions like AVX), there are key differences:

- **Programming Ease:** With SIMD, programmers must manually pack or vectorize data into wide registers and write loops. With SIMT, you simply write code for a *single independent thread*, and the hardware automatically scales it across thousands of threads.
- **Branching (Divergence):** If a code block has an if/else statement, SIMD struggles when different data lanes need different paths. In SIMT, threads can diverge and take different execution paths; the GPU handles this by temporarily masking off threads that aren't taking the active path and serializing the execution until they reconverge.

Streaming Multiprocessor (SM)

A Streaming Multiprocessor (SM) is the primary computing unit inside an NVIDIA GPU. If a GPU is a large factory, the SMs are the individual work teams inside that factory doing all the heavy lifting.

A streaming multiprocessor (SM) executes compute instructions on the GPU.

While a CPU has a few powerful cores built for sequential tasks, a GPU contains dozens or even over a hundred SMs designed to handle thousands of tasks at the exact same time.

What is Inside an SM?

An SM packs several specialized parts into a tiny area of silicon:

- **CUDA Cores:** The basic processing units that handle standard math and logic.
- **Tensor Cores:** Specialized mini-processors built to speed up AI, deep learning, and matrix math.

- **Warp Schedulers:** The managers that decide which tasks go to the cores during every clock cycle.
- **Register File:** Ultra-fast, tiny local memory storage used by active tasks.
- **Shared Memory / L1 Cache:** On-chip memory that lets nearby tasks share data quickly without traveling to the main GPU memory.

How Does an SM Process Data?

GPUs do not run single threads of code by themselves. Instead, they use a system called **SIMT** (Single Instruction, Multiple Threads):

- **Warps:** The SM groups threads into bundles of 32 called a **warp**.
- **Scheduling:** The warp scheduler gives the same instruction to all 32 threads in that warp to run at the same time.
- **Concurrency:** A single SM can manage multiple warps at once. If one warp is waiting to fetch data from memory, the SM instantly switches to another ready warp to keep the compute units busy.

Why Do SMs Matter?

When NVIDIA releases a new GPU generation, performance goes up mostly because they add more SMs to the chip, or because they make each individual SM smarter and faster (like adding newer generations of Tensor Cores). For instance, an NVIDIA Ampere A100 has 108 SMs, while a newer Hopper H100 has 132 SMs.

SMs are fundamental GPU execution resources responsible for running large numbers of GPU threads.

Synchronization and Communication

Parallel computing requires different processing resources to **coordinate and exchange information**.

For example, if eight GPUs perform distributed AI training, each GPU may calculate gradients using different training data. Those gradients need to be communicated and synchronized before the next training step.

This introduces an important AI infrastructure concept:

Compute performance alone is not enough.

At scale, performance depends heavily on:

GPU compute + GPU memory + memory bandwidth + inter-GPU communication + networking + software efficiency.

Technologies and communication libraries in the NVIDIA ecosystem are therefore important because GPUs must exchange large amounts of data efficiently during distributed AI workloads.

NVLink and High-Speed GPU Communication

When parallel workloads span multiple GPUs, **communication bandwidth and latency** become critical.

NVIDIA **NVLink** provides high-bandwidth connectivity between supported GPUs and other components in compatible platforms. Depending on the platform architecture, NVLink-based fabrics can enable GPUs to exchange data much faster than would be practical through conventional host-mediated communication paths.

This becomes particularly important for **tensor parallelism and distributed training**, where GPUs frequently exchange intermediate results.

The important infrastructure relationship is:

More GPUs ≠ automatically proportional performance.

If GPUs spend excessive time waiting for communication or synchronization, scaling efficiency decreases.

Collective Communication

Distributed AI workloads frequently require groups of GPUs to exchange information collectively.

Common operations include:

All-Reduce — combines values across GPUs and distributes the result back to participants.

All-Gather — gathers portions of data from multiple GPUs so participating GPUs obtain the combined data.

Reduce-Scatter — reduces data across GPUs and distributes portions of the resulting data.

These operations are particularly important in distributed deep-learning training.

NVIDIA's **NCCL (NVIDIA Collective Communications Library)** provides optimized collective communication primitives for NVIDIA GPUs.

A useful exam relationship is:

CUDA → GPU computation

NCCL → Multi-GPU collective communication

Why Networking Matters to Parallel Computing

Once a workload expands beyond a single GPU or server, **communication becomes part of the performance equation.**

Consider training a model across 64 GPUs. The GPUs may calculate extremely quickly, but if they frequently wait for data from other GPUs, overall training performance suffers.

This is why AI infrastructure focuses not only on accelerator performance but also on **memory bandwidth, interconnect bandwidth, network latency, network throughput, topology, and collective communication efficiency**.

This gives an important certification principle:

A distributed AI workload is only as efficient as the interaction between computation, memory, communication, and software.

Why Is Parallel Computing Important for AI?

Parallel computing is important for AI because neural networks require enormous numbers of matrix and tensor operations, many of which can execute independently and simultaneously. GPUs exploit this parallelism by running large numbers of threads concurrently. For models that require more resources than a single GPU provides, techniques such as data parallelism, tensor parallelism, and pipeline parallelism distribute workloads across multiple GPUs. Efficient communication and synchronization then become critical to maintaining scalability.

***Parallel computing** means dividing a computational workload into smaller operations that can execute simultaneously. It is fundamental to AI because neural networks perform large amounts of matrix and tensor computation that can be parallelized efficiently.*

NVIDIA GPUs exploit parallel computing by executing large numbers of threads across GPU execution resources such as Streaming Multiprocessors. CUDA provides the programming model for GPU computation. When workloads span multiple GPUs, techniques such as data, tensor, and pipeline parallelism are used, while technologies and libraries such as NVLink and NCCL help support efficient communication between GPUs.
